EECS 482
Introduction to operating systems

OS history, processes

Peter Chen
University of Michigan

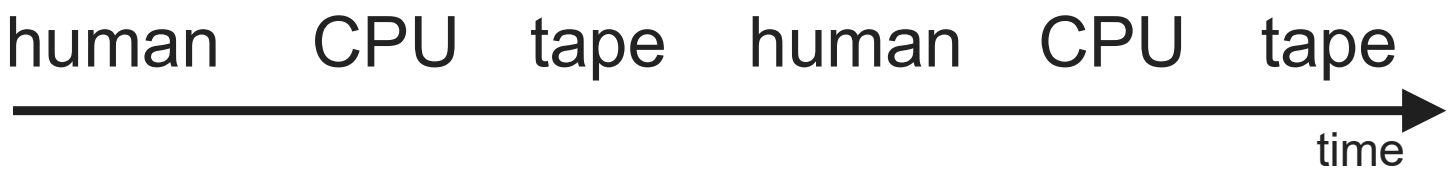
History of operating systems

- Computing hardware was very expensive
- OS were very simple, then became more sophisticated to use the hardware more efficiently

MIDAC (1953)
https://cse.engin.umich.edu/about/history/

Single operator at console

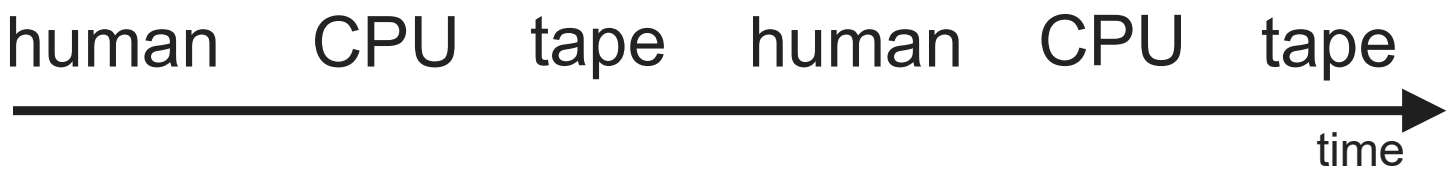
- Goal: basic functionality
- OS is library of standard services



- One thing happening at a time
  - + Interactive
  - - Poor utilization of hardware
- How to improve utilization?

Batch processing

- Goal: Improve utilization by removing user interaction



- Non-interactive: submit job to queue, wait for completion
- OS is library of standard services + batch monitor
- Protection becomes an issue
  - Must ensure batch monitor is not corrupted by program
  - Why wasn't this an issue for single operator at console?

Time sharing

- Goals: improve utilization by overlapping CPU and I/O from different jobs; restore interactivity for humans
- Insight: switch between processes while waiting for I/O or user

P_1: human CPU tape
P_2: CPU tape human
P_3: tape human CPU

- OS becomes more complex
  - Multiple jobs running concurrently
  - Multiple I/Os can take place simultaneously
  - Must protect jobs from each other

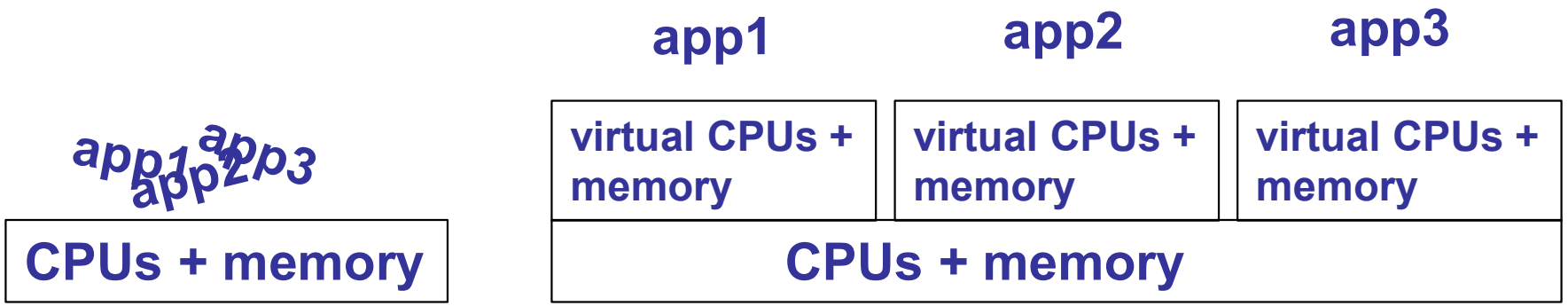
How to deal with complexity?

- Operating systems became complex to use hardware more efficiently
  - Multiple users, programs, I/O devices, etc.
  - Originally for efficient use of H/W, but useful even now

- How do we manage complexity in big programs?

Process abstraction

- Separate mix of activities running on a computer into several independent tasks



What is a process?

- Formal definition: one or more threads in an address space
- Informal definition: a program in execution
  - Note: process != program
- Analogy: a play being acted out
  - A program is the script of the play

Thread

- A sequence of executing instructions
- Active
- Analogy: a role in the play

Address space

- The memory data used by the threads
- Passive: acted on by the threads
- Analogy: the objects/props on the stage

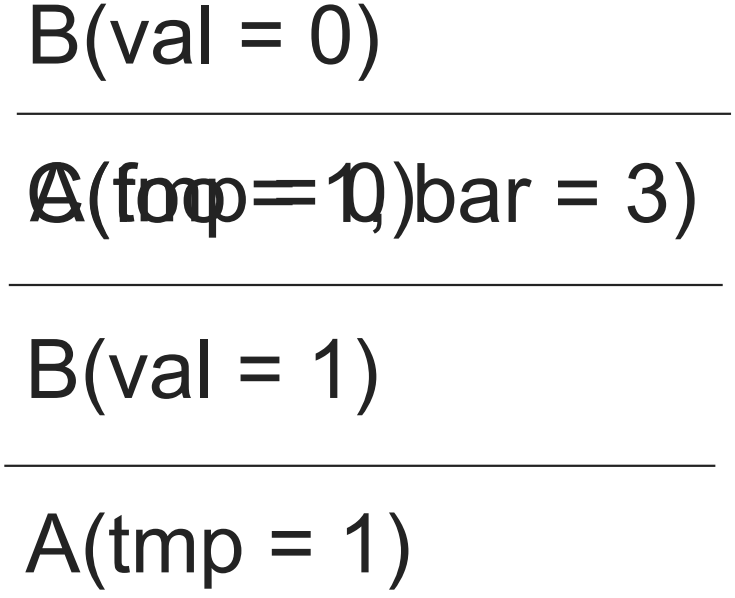
What memory data is used by your program as it runs?

Review: the call stack

```
A(int tmp) {
    B(tmp);
}

B(int val) {
    C(val, val + 2);
    A(val - 1);
}

C(int foo, int bar) {
    int v = bar - foo;
}
```



Per-thread state vs. shared state

- Code
  + code pointer (program counter)
- Stack
  + stack pointer
- Data segment (heap + static variables)

- Per-thread state: which state is private to each thread as it executes instructions and calls functions?

- Other state is shared:
  - Data segment

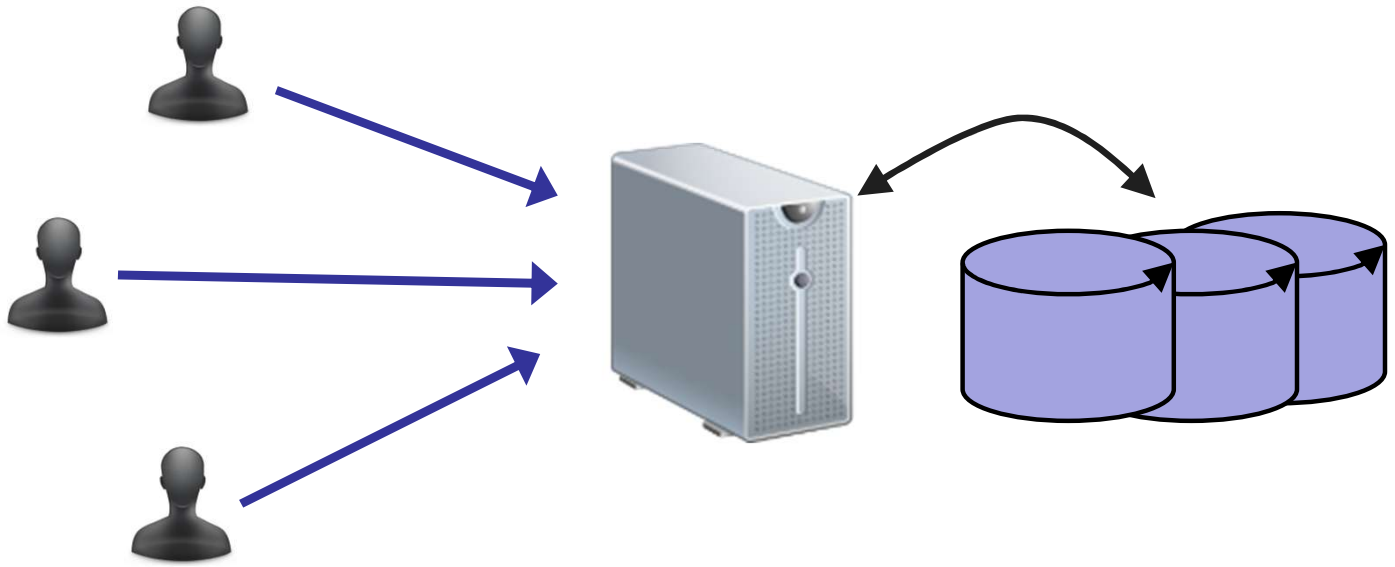
Upcoming topics

- Thread: unit of concurrency
  - How multiple threads cooperate to accomplish a task
  - How multiple threads can share limited number of CPUs

- Address space: unit of state partitioning
  - How do address spaces share single physical memory?
    - » Efficiently
    - » Flexibly
    - » Safely

Why do we need threads?

- Example: uni-processor web server
  - Receives multiple simultaneous requests
  - Reads web pages from disk to satisfy each request



Option 1: Handle one request at a time

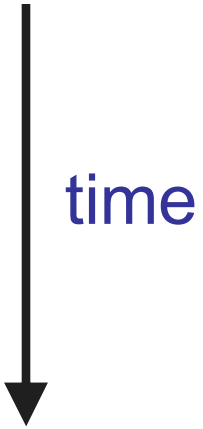
Request 1 arrives

Server receives request 1

Server starts disk I/O 1a and waits for it to complete

Request 2 arrives

Server must finish request 1 before handling request 2



- Easy to program, but slow
  - Can't overlap disk requests with computation
  - Can't overlap either with network sends and receives

Option 2: Asynchronous I/O (event-driven)

- Issue I/Os, but don't wait for them to complete

  Request 1 arrives
  Server receives request 1
  Server starts disk I/O 1a (but does not wait for it to complete)
  Request 2 arrives
  Server receives request 2
  Server starts disk I/O 2a
  Request 3 arrives
  Disk I/O 1a finishes
  Continue to process request 1

- Fast, but hard to program
  - Lots of extra state to remember at each point
  - What requests are being served, what stage they're in
  - What disk I/Os are outstanding; which requests they belong to

Option 3: Multi-threaded web server

- One thread per request
  - Thread issues disk (or network) I/O, then waits for it to finish
  - If some threads are blocked on I/O, other threads can still run
  - Where is the state of each request stored?

Thread 1	Thread 2	Thread 3
Request 1 arrives		
Receive request 1		
Start disk I/O 1a		
	Request 2 arrives	
	Receive request 2	
	Start disk I/O 2a	
		Request 3 arrives
		Receive request 3
Disk I/O 1a finishes		
Continue processing request 1		

Benefits of threads

- Thread manager takes care of sharing the CPU
  - One thread can issue blocking I/O, while other threads can still progress
  - Private state for each thread
- Applications get a simpler programming model
  - The illusion of a dedicated CPU per thread
- Downsides compared to event-driven model?

When are threads useful?

- Multiple things happening at once
- Usually some slow resource
  - Network, disk, user, …

- Examples:
  - Network server (e.g., web server)
  - Controlling a physical system (e.g., airplane controller)
  - Window system
  - Parallel programming

Ideal scenario

- Split computation into threads
- Threads run independently of each other
  - Divide and conquer works best if divided parts are independent

- How practical is thread independence?

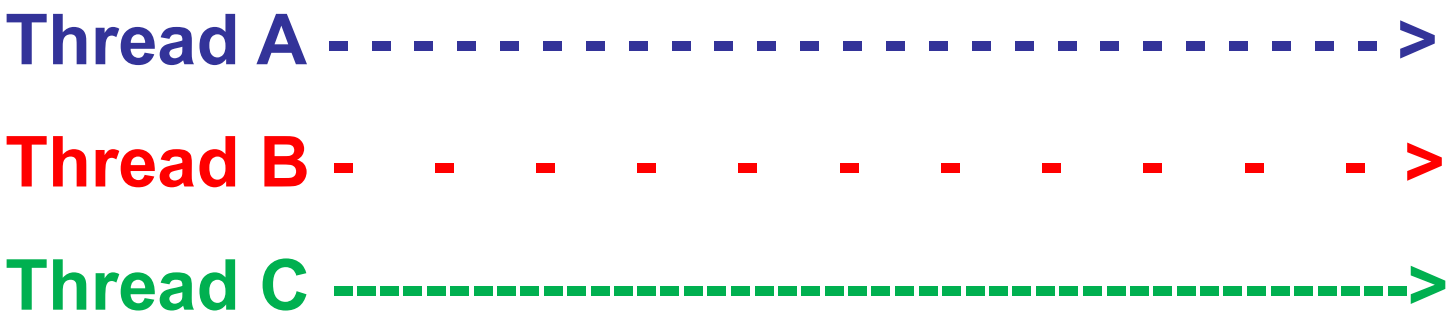
Completely independent threads?

- Example 1: Microsoft Word
  - One thread formats document
  - Another thread spell checks document
- Example 2: Desktop computer
  - One thread compiles EECS 482 project
  - Another thread runs Minecraft
- Two types of sharing: application resource or hardware resource

Cooperating threads

Problem:

- Speed of each processor is unpredictable
- Ordering of events across threads is non-deterministic



- Consequences:
  - Many possible global ordering of events
  - Different orderings may produce different results

Non-deterministic ordering → Non-deterministic results

- Printing example

Thread 1	Thread 2
Print ABC	Print 123

- What's being shared between these threads?
- Possible outputs:
- Impossible outputs:

- Ordering within each thread is sequential
- Many ways to merge per-thread order into a global order

Non-deterministic ordering → Non-deterministic results

- Assignment example (x is initially 0)

Thread A	Thread B
x = 1	x = -1

  - What's being shared between these threads?
  - Possible results:
  - Impossible results:

Non-deterministic ordering → Non-deterministic results

- Arithmetic example (y is initially 10)

Thread A	**Thread B**
x = y + 1	y = y * 2

  - What's being shared between these threads?

  - Possible results: